DISCIPLINA DE DESENVOLVIMENTO FRONTEND

APOSTILA

Ferramentas de Desenvolvimento Moderno

Módulo	1 - Fundamentos de Front-end
Entregável	Repositório GitHub configurado

Objetivos da aula

Ao final desta aula, o aluno sera capaz de:

- Configurar um ambiente de desenvolvimento front-end completo e profissional
- Usar npm para gerenciar dependências em projetos JavaScript
- Aplicar o fluxo de trabalho com Git e GitHub em projetos em equipe
- Aumentar a produtividade no VS Code com extensões e snippets
- Configurar ESLint e Prettier para garantir qualidade e padrão de código
- Configurar e inciar servidor de desenvolvimento com vite

Sumário

1. Cronograma da Aula....3
2. Node.js e npm....4
2.1. O que e o Node.js no contexto front-end....4
2.2. npm — Node Package Manager....4
Comandos essenciais....4
Anatomia do package.json....4
dependencies vs devDependencies....5
Exercicio 2.1 — Primeiro projeto npm....5
3. Terminal para Front-end....6
3.1. Comandos de navegação essenciais....6
3.2. Terminal integrado no VS Code....6
4. Git e GitHub na Prática....7
4.1. Fluxo básico revisado....7
4.2. Trabalhando com branches....7
4.3. O arquivo .gitignore....8
4.4. Boas práticas de mensagens de commit....8
5. VS Code Produtivo....9
5.1. Extensões essenciais....9
5.2. Alguns atalhos úteis do VS Code....9
6. ESLint e Prettier....10
6.1. Linter vs Formatter — não é a mesma coisa....10
6.2. Instalando e configurando o ESLint....10
6.3. Adicionando o comando ao package.json....10
6.4. Instalando e configurando o Prettier....11
6.5. Introdução ao Husky + lint-staged....12
7. Vite.js — Ferramenta de Build Moderna....13
7.1. Por que o Vite existe?....13
7.2. Instalação e uso básico....13
Templates disponíveis....14
7.3. Estrutura de um projeto Vite....14
O arquivo vite.config.ts....15
7.4. Adicionando Vite a um projeto existente....15
Passo 1 — Inicializar o npm e instalar o Vite....15
Passo 2 — Configurar os scripts no package.json....15
Passo 3 — Configurar o vite.config.js....16
Passo 4 — Executar o projeto....16
7.5. Vite vs Webpack — Comparativo....17
8. Atividade — Repositório....18
8.1 Detalhes da atividade....18
8.2 Prazo e forma de entrega....18
9. Referências e Materiais Complementares....19
9.1. Documentação oficial....19

1. Cronograma da Aula

A aula esta dividida em seis blocos tematicos, cada um com teoria, demonstracao e exercicio prático. O tempo e estimado; adapte conforme o ritmo da turma.

Bloco	Tema
1	Node.js e npm
2	Terminal para front-end
3	Git e GitHub na prática
4	VS Code produtivo
5	ESLint/Prettier/Husky

Atenção — Preparação

Precisa instalar o Node.js LTS (versão atual) pelo site oficial nodejs.org.

No Windows, recomende o uso do nvm-windows para evitar problemas de permissão e PATH.

2. Node.js e npm

2.1. O que e o Node.js no contexto front-end

Node.js é um runtime JavaScript que executa fora do navegador. No desenvolvimento front-end, ele não é usado para rodar a aplicação em produção — ele fornece o ecossistema de ferramentas: bundlers, servidores de desenvolvimento, linters, transpiladores e gerenciadores de pacotes.

2.2. npm — Node Package Manager

O npm é o gerenciador de pacotes padrão do Node.js. Permite instalar, remover e versionar bibliotecas de terceiros, além de definir scripts de automação para o projeto.

Comandos essenciais

```
# Inicializar um novo projeto
npm init -y

# Instalar uma dependência de produção
npm install axios

# Instalar uma dependência de desenvolvimento
npm install --save-dev eslint

# Remover um pacote
npm uninstall axios

# Executar um script definido no package.json
npm run dev

# Instalar todas as dependências listadas no package.json
npm install
```

Anatomia do package.json

O arquivo package.json e o "passaporte" do projeto. Ele registra tudo o que o projeto precisa para funcionar:

```
{
  "name": "meu-projeto",
  "version": "1.0.0",
  "scripts": {
    "dev":    "vite",
    "build":  "vite build",
    "lint":   "eslint . --ext .js,.ts",
    "format": "prettier --write ."
```

```
  },
  "dependencies": {
    "axios": "^1.6.0"
  },
  "devDependencies": {
    "eslint":   "^8.57.0",
    "prettier": "^3.2.5",
    "vite":     "^5.2.0"
  }
}
```

dependencies vs devDependencies

dependencies	Bibliotecas necessarias em produção (ex: axios, react)
devDependencies	Ferramentas usadas só durante o desenvolvimento (ex: eslint, vite)
node_modules/	Pasta gerada pelo npm com os arquivos dos pacotes — NUNCA commitar
package-lock.json	Registra as versões exatas instaladas — SEMPRE commitar

Exercício 2.1 — Primeiro projeto npm

Ex. 2.1 Criando seu primeiro projeto npm	
Descrição	Crie uma pasta chamada aula-npm. Inicialize um projeto com npm init -y. Instale o pacote date-fns como dependência. Adicione “type”: “module” ao seu package.json. (Isso é necessário para utilizar o import no padrão ECMASCRIPT6). Crie um arquivo index.js que importa a função format do date-fns e imprime a data atual formatada (ex: '19 de Março de 2026'). Execute com node index.js.
Objetivo	Entender o ciclo install -> importar -> usar e observar as mudanças no package.json antes e depois da instalação.

Dica	`Use:`
	`import { format } from 'date-fns';`
	`import { ptBR } from 'date-fns/locale';`
	
	`const data = new Date(); // Ex: 2024-03-15`
	`const dataFormatada = format(data, "dd 'de' MMMM 'de' yyyy",`
	`{ locale: ptBR });`
	
	`console.log(dataFormatada); // "15 de março de 2024"`

3. Terminal para Front-end

O terminal e a interface de linha de comando que usamos para executar scripts, navegar pelo sistema de arquivos, rodar servidores locais e interagir com o Git. Dominar o terminal é uma habilidade essencial para qualquer desenvolvedor front-end profissional.

3.1. Comandos de navegação essenciais

Comando	O que faz
`pwd`	Mostra o diretório atual (print working directory)
`ls / dir`	Lista arquivos e pastas (Unix / Windows)
`cd nome-da-pasta`	Entra em uma pasta
`cd ..`	Volta uma pasta acima
`mkdir nome`	Cria uma nova pasta
`touch arquivo.js`	Cria um arquivo vazio (Unix)
`rm -rf pasta/`	Remove pasta e todo seu conteúdo (cuidado!)
`clear`	Limpa a tela do terminal
`Ctrl + C`	Interrompe o processo atual (ex: servidor rodando)

3.2. Terminal integrado no VS Code

O VS Code possui um terminal integrado que pode ser aberto com Ctrl + ` (acento grave). Você pode ter múltiplos terminais abertos simultaneamente — muito útil para rodar o servidor de desenvolvimento em um e executar comandos em outro.

- Abrir novo terminal: Ctrl + ` ou menu Terminal > Novo Terminal
- Dividir terminal: ícone de split no canto superior do painel
- Alternar entre terminais: clique no nome na lista lateral

Windows vs Unix

No Windows, o terminal padrão e o PowerShell ou CMD. Comandos como touch e ls não existem nativamente.

Solução recomendada: instale o Git for Windows, que inclui o Git Bash — um terminal Unix no Windows.

Alternativa: use o WSL2 (Windows Subsystem for Linux) para um ambiente Linux nativo.

4. Git e GitHub na Prática

Git é um sistema de controle de versao distribuído. Ele registra o histórico de alterações do código, permitindo colaborar com outras pessoas, reverter mudanças e trabalhar em funcionalidades paralelas sem conflito.

4.1. Fluxo básico revisado

```
# 1. Inicializar repositório local
git init

# 2. Conectar ao repositório remoto (GitHub)
git remote add origin https://github.com/usuario/repo.git

# 3. Selecionar arquivos para o commit
git add .                   # todos os arquivos
git add src/index.js        # arquivo específico

# 4. Criar o commit com mensagem descritiva
git commit -m "feat: adiciona componente de cabecalho"

# 5. Enviar para o GitHub
git push origin main
```

4.2. Trabalhando com branches

Branches (ramificações) permitem desenvolver funcionalidades de forma isolada sem afetar o código principal. O fluxo padrão em equipes e o Feature Branch Workflow:

```
# Criar e entrar em um novo branch
git checkout -b feature/novo-cabecalho

# Verificar em qual branch você esta
git branch

# Voltar para a branch principal
git checkout main

# Fazer merge do feature branch
git merge feature/novo-cabecalho

# Deletar branch após merge
git branch -d feature/novo-cabecalho
```

4.3. O arquivo .gitignore

O .gitignore lista arquivos e pastas que o Git deve ignorar — ou seja, que nunca serão enviados para o repositório remoto.

```
# .gitignore para projetos front-end

# Dependências (geradas pelo npm install)
node_modules/

# Arquivos de build
dist/
build/
.next/

# Variáveis de ambiente (senhas, chaves de API)
.env
.env.local
.env.production

# Arquivos de sistema
.DS_Store       # macOS
Thumbs.db       # Windows

# Logs
*.log
npm-debug.log*
```

4.4. Boas práticas de mensagens de commit

Mensagens de commit descritivas facilitam a leitura do histórico do projeto. O padrão Conventional Commits é amplamente adotado:

feat:	Nova funcionalidade: feat: adiciona botão de login
fix:	Correção de bug: fix: corrige erro ao enviar formulario
docs:	Documentação: docs: atualiza README com instruções de setup
style:	Formatação sem mudança de lógica: style: aplica prettier
refactor:	Refatoração sem nova funcionalidade: refactor: extrai função
chore:	Tarefas de manutenção: chore: atualiza dependências

5. VS Code Produtivo

O Visual Studio Code é o editor mais utilizado por desenvolvedores front-end. Sua produtividade aumenta consideravelmente quando você aprende a usar extensões, atalhos e snippets.

5.1. Extensões essenciais

Extensão	Para que serve
ESLint	Mostra erros de linting diretamente no editor com sublinhado vermelho
Prettier	Formata o código automaticamente ao salvar
GitLens	Mostra quem alterou cada linha do código e quando (git blame inline)
Error Lens	Exibe mensagens de erro ao lado da linha problematica
Path Intellisense	Autocompleta caminhos de arquivos ao importar módulos
Auto Rename Tag	Renomeia a tag HTML de fechamento ao alterar a de abertura

5.2. Alguns atalhos úteis do VS Code

Ctrl + P	Abrir qualquer arquivo pelo nome
Ctrl + Shift + P	Paleta de comandos (acesso a tudo)
Alt + Click	Multi-cursor: editar varios pontos ao mesmo tempo
Alt + Seta cima/baixo	Mover linha para cima ou para baixo
Ctrl + D	Selecionar a próxima ocorrencia do mesmo texto
F2	Renomear símbolo (variável, função) em todo o projeto
F12	Ir para a definição de uma função ou variável
Alt + F12	Peek definition: ver definição sem sair do arquivo
Ctrl + `	Abrir/fechar o terminal integrado

6. ESLint e Prettier

6.1. Linter vs Formatter — não é a mesma coisa

Uma confusão muito comum e tratar ESLint e Prettier como equivalentes. Eles tem papeis complementares e diferentes:

ESLint (Linter)	Analisa o código em busca de erros de lógica, variáveis não usadas, padrões proibidos. Encontra PROBLEMAS.
Prettier (Formatter)	Reformata o código para um estilo visual consistente: espaços, aspas, ponto e vírgula. Define ESTETICA.
Uso combinado	ESLint encontra o problema, Prettier formata o resultado. Os dois trabalham juntos.

6.2. Instalando e configurando o ESLint

```
# Instalar o ESLint interativamente – (forma recomendada)
npm init @eslint/config

# Ou instalar manualmente
npm install --save-dev eslint

# Rodar o ESLint em todo o projeto
npx eslint .

# Rodar o ESLint em todo o projeto
npx eslint .

# Corrigir automaticamente o que for possível
npx eslint . --fix
```

6.3. Adicionando o comando ao package.json

Adicionar ao package.json a linha “lint”: “eslint .”

```
Scripts : {
  “lint”: “eslint .”
}
```

Ao rodar *npm run lint* no terminal, o eslint irá realizar a varredura do código em busca de problremas

6.4. Instalando e configurando o Prettier

```
# Instalar o Prettier
npm install --save-dev prettier

# Instalar a integração ESLint + Prettier (evita conflitos)
npm install --save-dev eslint-config-prettier

# Formatar todos os arquivos
npx prettier --write "**/*.{js,ts,css}"

# Verificar sem alterar
npx prettier --check "**/*.{js,ts,css}"
```

O arquivo .prettierrc define as preferências de formatação:

```
{
  "semi":          true,
  "singleQuote":   true,
  "tabWidth":      2,
  "trailingComma": "es5",
  "printWidth":    80,
  "arrowParens":   "always"
}
```

Adicione eslint-config-prettier ao .eslintrc.config.js para desativar regras do ESLint que conflitam com o Prettier. No arquivo abaixo já foram incluídas algumas implementações para ignorar a pasta node_modules, .husky, dist e build (pastas que são geradas automaticamente ou possuem código de terceiros).

```
import js from "@eslint/js";
import globals from "globals";
import prettier from "eslint-config-prettier";

export default [
  { ignores: ["node_modules/", ".husky/", "dist/", "build/"] },
  js.configs.recommended,
  prettier,
  {
    files: ["**/*.{js,mjs,cjs}"],
    languageOptions: {
      globals: globals.browser,
      ecmaVersion: 2020,
      sourceType: "module"
    },
    rules: {
      "no-unused-vars": ["error", { "args": "none" }]
    }
  },
];
```

6.5. Introdução ao Husky + lint-staged

O Husky executa scripts automaticamente nos hooks do Git. O lint-staged aplica o ESLint e o Prettier apenas nos arquivos que estão sendo commitados, tornando o processo rápido.

```
# Inicializar um repositorio git
git init

# criar ou vincular um novo repositorio para os testes
git remote add origin https://github.com/usuario/repo.git

#Instalar dependências de dev
npm install -D husky lint-staged

#Inicializar o Husky no projeto
npx husky init

#Confirmar se o script prepare foi criado em package.json
#Deve existir algo assim em scripts:
prepare: "husky"

#Configurar o hook pre-commit em pre-commit:1
#Conteúdo recomendado:
npx lint-staged

#Configurar o lint-staged em package.json
#Exemplo:
"lint-staged": {
    "*.{js,ts}": [
      "eslint --fix",
      "prettier --write"
    ]
  }

#Garantir ignore correto de dependências em .gitignore
#Deve ter:
node_modules/

# Na pasta .huskie edite arquivo pre-commit. Adicione
npx lint-staged

# Testar se o hook está funcionando

Edite um arquivo JS
Faça git add .
Faça git commit -m "teste husky"
Se houver erro de lint/format, o commit deve ser bloqueado.
```

7. Vite.js — Ferramenta de Build Moderna

O Vite é uma ferramenta de build de nova geração desenvolvida por Evan You, o mesmo criador do Vue.js. Ele resolve um dos maiores problemas do desenvolvimento front-end moderno: a lentidão do servidor de desenvolvimento conforme o projeto cresce.

7.1. Por que o Vite existe?

Ferramentas como Webpack precisam processar e empacotar todos os módulos antes de iniciar o servidor de desenvolvimento. Em projetos grandes, isso pode levar dezenas de segundos a cada reinicialização.

O Vite resolve isso de duas formas principais:

Como o Vite funciona internamente

1. Pré-bundling de dependências: dependências externas (node_modules) são pré-processadas com esbuild — um bundler escrito em Go, 10 a 100 vezes mais rápido que bundlers JavaScript.
2. Módulos ES nativos em desenvolvimento: o código da aplicação é servido como ESM nativo ao navegador, sem bundle. O browser carrega os módulos sob demanda.
3. HMR ultrarrápido: Hot Module Replacement atualiza apenas o módulo alterado, independente do tamanho do projeto.

7.2. Instalação e uso básico

Criar um novo projeto com Vite é simples. O comando abaixo abre um assistente interativo para escolher o framework e a linguagem:

```
# Criar novo projeto com Vite (assistente interativo)
npm create vite@latest meu-projeto

# Ou especificando diretamente o template
npm create vite@latest meu-projeto -- --template react-ts

# Entrar na pasta e instalar dependências
cd meu-projeto
npm install

# ou adicionar o vite a um projeto existente
npm install -D vite # para projetos existentes é necessário criar o arquivo de
configuração vite.config.js:
```

```
// Conteudo vite.config.js
import { defineConfig } from 'vite'

export default defineConfig({
  plugins: [],
})

# adicionar os comando para iniciar o servidor e build (gera os arquivos para
produção)
// package.json → scripts :
  "dev": "vite",
  "build": "vite build"

# Iniciar o servidor de desenvolvimento
npm run dev

# Gerar build de produção
npm run build

# Pré-visualizar o build de produção localmente
npm run preview
```

Templates disponíveis

O Vite oferece templates oficiais para os principais frameworks e linguagens:

Template	Descrição
`vanilla`	JavaScript puro, sem framework
`vanilla-ts`	TypeScript puro, sem framework
`react`	React com JavaScript
`react-ts`	React com TypeScript (recomendado para a disciplina)
`vue`	Vue 3 com JavaScript
`svelte-ts`	Svelte com TypeScript

7.3. Estrutura de um projeto Vite

Um projeto Vite recém-criado com o template react-ts tem a seguinte estrutura:

```
meu-projeto/
├── public/              # Arquivos estáticos (copiados sem processamento)
│   └── vite.svg
├── src/                 # Código-fonte da aplicação
│   ├── assets/
│   ├── App.tsx
│   └── main.tsx         # Ponto de entrada
├── index.html           # HTML raiz (não fica em /public!)
```

```
├── package.json
├── tsconfig.json
└── vite.config.ts        # Configuração do Vite
```

O arquivo vite.config.ts

A configuração do Vite é mínima por padrão, mas altamente extensível via plugins:

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  server: {
    port: 3000,           // porta do servidor de desenvolvimento
    open: true,           // abre o browser automaticamente
    proxy: {              // redireciona chamadas de API para o back-end
      '/api': 'http://localhost:8080'
    }
  },
  build: {
    outDir: 'dist',       // pasta de saída do build
    sourcemap: true       // gera source maps para debug em produção
  }
})
```

7.4. Adicionando Vite a um projeto existente

Se você já possui um projeto HTML/CSS/JavaScript e deseja adicionar o Vite sem recriá-lo do zero, siga os passos abaixo. O Vite se integra facilmente a projetos existentes com poucos comandos.

Passo 1 — Inicializar o npm e instalar o Vite

```
# Na raiz do projeto existente, inicialize o npm (se ainda não tiver package.json)
npm init -y

# Instale o Vite como dependência de desenvolvimento
npm install --save-dev vite
```

Passo 2 — Configurar os scripts no package.json

```
# Adicione os scripts abaixo em package.json:
{
  "scripts": {
    "dev":     "vite",
```

```
    "build":   "vite build"
  }
}
```

Passo 3 — Configurar o vite.config.js

```
# ou adicionar o vite a um projeto existente
npm install -D vite

# para projetos existentes é necessário criar o arquivo de configuração
vite.config.js:

// Conteudo vite.config.js
import { defineConfig } from 'vite'

export default defineConfig({
  plugins: [],
})
```

Passo 4 — Executar o projeto

```
# Iniciar o servidor de desenvolvimento com HMR
npm run dev

# Gerar build otimizado de produção
npm run build

# Pré-visualizar o build localmente antes de publicar
npm run preview
```

7.5. Vite vs Webpack — Comparativo

Os dois são ferramentas de build amplamente utilizadas, mas com filosofias diferentes. O Vite prioriza a experiência de desenvolvimento; o Webpack prioriza controle e flexibilidade total.

Critério	Vite	Webpack
Filosofia	ESM nativo + HMR ultrarrápido	Bundling completo em todos os ambientes
Velocidade de inicialização	Muito rápido — não faz bundle em dev	Mais lento — faz bundle completo
Build de produção	Usa Rollup internamente	Bundle próprio otimizado
Configuração	Mínima — zero config na maioria dos casos	Extensível, porém mais verbosa
Ecossistema	Crescente — padrão com React, Vue, Svelte	Maduro — amplamente usado em produção
Curva de aprendizado	Baixa	Média a alta
Quando usar	Novos projetos, prototipagem, React/Vue	Projetos legados, configuração muito personalizada

8. Atividade — Repositório

8.1 Detalhes da atividade

Com base no conteúdo apresentado na aula, configure no site desenvolvido nas aulas anteriores as ferramentas:

1. Crie uma nova branch a partir da principal (main) com o nome feature-ferramentas-desenvolvimento;
2. Adicione npm;
3. Adicione o arquivo .gitignore ao projeto e configure para ignorar a pasta node_modules;
4. Adicione o ESLint;
5. Configure o comando do ESLint no package.json
6. Adicione Prettier;
7. Configure o comando do prettier no package.json
8. Adicione o husky ao projeto;
9. Configure o husky e lint-staged
10. Faça um teste de falha, fazendo o commit de um código incorreto.

8.2 Prazo e forma de entrega

Repasse o link do repositório github via whatsapp (pode postar no grupo ou enviar no privado) ou e-mail welerson.buganca@prof.sc.senac.br.

9. Referências e Materiais Complementares

9.1. Documentação oficial

- Node.js — nodejs.org/pt-br/docs
- npm CLI — docs.npmjs.com/cli/v10/commands
- Git — git-scm.com/book/pt-br/v2 (Pro Git Book em português)
- VS Code Snippets — code.visualstudio.com/docs/editor/userdefinedsnippets
- ESLint Getting Started — eslint.org/docs/latest/use/getting-started
- Prettier Install — prettier.io/docs/en/install.html
- Husky — typicode.github.io/husky

Disciplina de Desenvolvimento Frontend — Material didático de uso exclusivo